NODE JS

01#########

What is Node js

1. Node js is runtime environment for javaScript.
2. Node we can run JavaScript outside to the browser

Scripting Languages

Like server, windows, mac , linux etc

1. JavaScript run inside the browser.

Where we need JS outside browser

1. Web browser Run on client computer.
2. So Javascript do not handle things on server.
3. So we can not connect javascript with Database directly
4. Can not handle File system etc
5. So we have to run javascript on browser and Node help for this

Why need Node

1. Unified Language
2. High Performance
3. Good for Real-time Applications
4. Good for Data Streaming
5. Etc

How to use it full stack and FE technology

1. We make API in node js.
2. Integrate APIs with FE technology.
3. BE+FE = Full Stack.

Java (Programming Language)

History Of Node js

1. First Release :- May 27, 2009
2. Current Version: V24.21.0
3. Written in JS, C and c++

02#########

Node.js Setup on MacOS

Step 1: Download Node.js

1. Go to the official Node.js website:
2. 👉 https://nodejs.org/en
3. Click on **Download Node.js (LTS)** version.
4. *(LTS means Long Term Support – best for most users.)*

Scripting Languages

Step 2: Install Node.js

1. Open the downloaded file and complete the installation steps.

2. Once installed, Node.js and **npm (Node Package Manager)** will be available.

Check Installed Versions

Open **Terminal**.

Type the following commands to verify installation:

node -v

Mac OS

npm -v

This will show you the version numbers of Node.js and npm installed.

Node.js Setup on Windows

Step 1: Download Node.js

1. Go to:
2. 👉 https://nodejs.org/en
3. Click on the **Download Node.js (LTS)** version.

Step 2: Install Node.js

1. Run the downloaded setup file and complete the installation.
2. This installs both **Node.js** and **npm**.

Step 3: Check Installed Versions

1. Open **Command Prompt (CMD)**.
2. Run these commands:

Java (Programming Language)

node -v

npm -v

You should see version numbers if everything was installed correctly.

Install VS Code Editor

To write and run Node.js code smoothly, install **Visual Studio Code** (VS Code) — a lightweight code editor.

For macOS:

1. Visit: https://code.visualstudio.com/
2. Click **Download for macOS** and follow the install instructions.

Scripting Languages

For Windows:

1. Same site: https://code.visualstudio.com/
2. Click **Download for Windows**.

Once installed, open VS Code and start coding in JavaScript and Node.js easily.

3#########

Create Folder and File

Create Folder

On Desktop, create a folder manually (e.g., NodeApp)

or

Open Terminal and run:

mkdir NodeApp

cd NodeApp

Open Folder in VS Code

Using VS Code directly

Open **VS Code**

Go to **File > Open Folder**

Select your NodeApp folder

Using Terminal

code .

This opens the current folder in VS Code (if code command is configured).

Create a JavaScript File

1. Create a file named app.js

Node.js runs JavaScript files, so .js extension is required.

Scripting Languages

Write Some Basic Code

Basic Output

```
console.log(20 + 20); // Output: 40
```

Function Example

```
function fruit(item) {
```

```
console.log("Fruit name is " + item);
```

Java (Programming Language)

```
}
fruit("apple");
```

Variable Example

```
var a = 10;
var b = 20;

console.log(a + b); // Output: 30
```

Run Node.js Code

Steps:

1. Open terminal
2. Navigate to your project folder (if not already)
3. Run:

```
node app.js
```

File System Example (Just to Show)

```
var fs = require("fs"); // Import file system module
```

Operating Systems

```
fs.writeFileSync("anil.txt", "My name is Anil");
```

This creates a file anil.txt and writes the content inside it.

REPL (Read Eval Print Loop)

1. Just type node in terminal to enter REPL mode
2. Then type JS code line by line

> 2 + 2

4

> .exit

4#########

What Are Modules and Their Types?

Core Modules

1. Core modules are already available in Node.js.
2. You do not need to install them.

```
const fs = require("fs");
```

Learn Scripting

```
const os = require("os"); // import os module


fs.writeFileSync("dummy.txt", "trying with modules");
// Create a file with name and write content in it


console.log(os.platform()); // Print the platform of the OS
console.log(os.hostname()); // Print the hostname of the OS
console.log(os.cpus()); // Print the CPU information of the OS
```

Third-party Modules

You can install them from third party.

Custom Modules

You create them by yourself.

Earn a CS Degree

What Are Core Modules?

1. Core modules are modules that are already included in Node.js.
2. No need to install them separately. Just use require() to import them.

What Are Global Objects?

1. Global object is an object that is available in all modules by default.
2. You do not need to import or require it to use it.

You can import console forcefully:

Learn Java

```
const { log } = require("console"); // import console module


log("custom log");
```

Difference Between console.log in JS and Node.js

The difference is:

1. In **JavaScript** → console.log prints on the browser console.
2. In **Node.js** → console.log prints on the terminal.

5########

Why Do We Need a Server?

JavaScript by default runs in the browser.

But when we want to run JavaScript on the server, we need Node.js.

Now, if you write some code in Node.js — how can you access that code in the browser?

It's important to understand this:

If you want to see your Node.js output in the browser, you need to set up a server.

Write Code to Create a Server

Node.js provides a built-in package called http to create servers.

You don't need to install it separately.

Here's the basic code to create a server:

```
const http = require("http");

http.createServer().listen(4800);
```

How to Run the Code

Open your terminal and run the file using:

node basics.js

How to Check the Output in the Browser

Open your browser and type:

http://localhost:4800

You can also check the response in:

Inspect → Network tab

Understanding Request and Response Parameters

If you want your server to return some data to the browser, this is how it works:

1. The **client** (browser) sends a request.
2. The **server** responds to that request.

In Node.js, you can handle this using the http.createServer() method, which accepts a callback function with request and response parameters.

```
http
.createServer((req, resp) => {
resp.write("<h1>Hello, this is Anil Sidhu</h1>");
resp.end("Hello"); // Ends the response
})
.listen(4800);
```

Now, when you visit http://localhost:4800, you will see the message in the browser.

6#########

What is an External Package?

An external package is a package we install from a third party (someone else wrote it).

We use it to add **extra functionality** to our project without writing everything ourselves.

Example:

nodemon — a package that automatically restarts the server when code changes.

Without this, we would have to stop and start the server manually after every change.

How to Install an External Package

Example: Install the colors package from https://www.npmjs.com/package/colors

Open the terminal and run:

npm install colors

Try Cloud Tools

or

npm i colors

Example of Using an External Package

```
const colors = require("colors");


console.log(colors.red("Hi, this is Anil Sidhu"));
console.log(colors.green("Hi, this is Anil Sidhu"));
console.log(colors.yellow("Hi, this is Anil Sidhu"));
```

To run this code, type in the terminal:

node colors.js

Follow Stars

What is package.json?

package.json is a file that contains details about your project, such as:

1. Project name
2. Project version
3. External packages used (dependencies)
4. Project description
5. Author
6. License

What is package-lock.json?

package-lock.json is a file that contains detailed information about the exact versions of every installed package **and its dependencies** (internal packages).

Learn Scripting

It ensures that everyone working on the project installs **the same package versions**.

What is node_modules?

node_modules is a folder where all installed external packages are stored.

How to Create a package.json File

Run this command in the terminal:

npm init

Earn a CS Degree

You will be asked to fill in some details:

package name: node-2026

version: (1.0.0)

description: node js tutorial in hindi 2026

entry point: (index.js)

git repository:

keywords:

author: Anil Sidhu

license: (ISC)

Important Notes

1. External packages are stored inside the node_modules folder.
2. **You should not push node_modules to GitHub** — because it is a very large folder.
3. Other developers can easily install the required packages by running:

Try Cloud Tools

npm install

This will read package.json and package-lock.json and install everything needed.

7#########

Why do we need the nodemon package?

When you're working with a Node.js server, every time you make a change in your server file, you have to:

1. **Manually stop** the server.
2. **Restart** it to see the changes.

This process becomes annoying during development

Solution: nodemon

1. nodemon automatically **watches your files**.
2. It **restarts the server** automatically whenever you make changes.

3. Saves **time** and improves **developer experience**.

How to Install Nodemon

Open your terminal and run:

npm install nodemon

How to Use Nodemon

nodemon app.js

Now the server will restart automatically every time you make changes to app.js

Can we create more than one server?

Yes, we can run multiple servers in Node.js — just make sure they run on **different ports**.

10#########

What is a Server Response?

1. When a **client** (like a browser) sends a request to a server (e.g., by visiting google.com), the **server processes the request** and sends back a **response**.
2. This response can be **HTML**, **JSON**, **plain text**, files, or anything else.

Java (Programming Language)

Example:

When you visit **google.com**, the server sends back HTML that builds the **Google homepage** on your screen.

Basic Example of Server Response

const http = require("http");

```
http
  .createServer((req, resp) => {
    resp.setHeader("Content-Type", "text/html"); // Tell browser the response is HTML
    resp.write("<h1>Hello Anil Sidhu</h1>");
    resp.end(); // End the response
  })
  .listen("4800");
```

How to run:

1. Save as app.js
2. Run: nodemon app.js or node app.js
3. Open your browser and go to: http://localhost:4800

Passing Variables and Functions in Response

You can also include **variables**, **functions**, or **dynamic values** (like current time):

```
const http = require("http");

const age = 29;
```

Web Design & Development

```
http
  .createServer((req, resp) => {
    resp.setHeader("Content-Type", "text/html");

    resp.write(`
```

```
    <html>
      <head>
        <title>Code Step by Step</title>
      </head>
      <body>
        <h1>Hello Anil Sidhu</h1>
        <h2>Age: ${age}</h2>
        <h2>Current Date & Time: ${Date()}</h2>
      </body>
    </html>
  `);

  resp.end();
})
.listen(4800);
```

1. resp.setHeader("Content-Type", "text/html"): Tells the browser that we're sending back HTML.
2. Template literals (` `): Used to inject JavaScript variables like age and Date() into the HTML.

11#########

Create Server in simpleAPI.js

We'll use Node.js http module to create a basic server.

Learn to Code

Define Static API Data

```
const userData = [
{
name: "Anil",
age: 30,
email: "anil@test.com",
},
{
name: "sam",
age: 20,
email: "sam@test.com",
},
{
name: "peter",
```

Learn Java

```
age: 30,
email: "peter@test.com",
},
];
```

This is sample static data to simulate API response.

Use This Data in Server Response

```
const http = require("http");
```

```
http
.createServer((req, resp) => {
resp.setHeader("Content-Type", "application/json"); // Set the content type to JSON
Learn Scripting
resp.write(JSON.stringify(userData)); // Convert the userData array to a JSON string
resp.end(); // End the response
})
.listen(6000);
```

How to Test the API

Run the file using:

nodemon simpleAPI.js

Open browser and go to:

Browse Art Prints

http://localhost:6000

Test API using Thunder Client

You can also use Thunder Client (VS Code extension) or Postman:

1. Method: GET
2. URL: http://localhost:6000

You will get the JSON response.

13#########

What is Command Line Input?

Sometimes, we want to pass values **directly while running a file** from the terminal — like a username or a dynamic port number.

Create File: command-line-input.js

nodemon command-line-input.js anil sidhu

This runs the file and passes "anil" and "sidhu" as input arguments.

How to Get Command Line Input

```
const arg = process.argv;


console.log("-------", arg);
```

Output:------- [

'C:\\Program Files\\nodejs\\node.exe',

'C:\\Users\\sheet\\OneDrive\\Desktop\\turorial\\node 2026\\command-line-input.js',

'anil',

'sidhu'

]

 process.argv is an array:

1. Index 0 → Node path
2. Index 1 → File path
3. Index 2 onwards → Actual input values

Learn Java

Access Individual Input

```
console.log("-------", arg[3]);
```

Output:

------- sidhu

Note: Space-separated values are stored as separate elements in the array.

Example Use Case: Dynamic Port

```
const http = require("http");
const arg = process.argv;
const port = arg[2];

http
  .createServer((req, resp) => {
    resp.write("test input from cmd");
    resp.end();
  })
  .listen(port);
```

Run this from Terminal:

nodemon command-line-input.js 4100

This will start a server

14#########

Why load an HTML file with fs?

1. Instead of writing HTML as a string in resp.write(), it's more scalable and readable to keep HTML in a separate .html file.
2. This approach is cleaner, especially for large or structured HTML.

Computer Drives & Storage

Server File (web.js)

```
const http = require("http");


http.createServer((req, resp) => {}).listen(3200);
```

1. Creates a valid HTTP server.
2. Listens on port 3200.

HTML File (web.html)

```
<!DOCTYPE html>
```

Operating Systems

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Web page</title>
  </head>
```

```
 <body>
  <h1>Web Page Heading</h1>
  <input type="text" />
  <button>Click</button>
 </body>
</html>
```

1. Well-formed HTML structure.
2. Proper title, meta tags, and UI elements.

Read and Load HTML File Using fs Module

```
const http = require("http");
```

Web Design & Development

```
const fs = require("fs");


http
 .createServer((req, resp) => {
  fs.readFile("html/web.html", "utf-8", (err, data) => {
   if (err) {
    resp.writeHead(500, {
     "Content-Type": "text/plain",
    });
    resp.write("internal server error");
    resp.end();
    return;
   }
```

```
    resp.writeHead(200, {
      "Content-Type": "text/html",
    });
    resp.write(data);
    resp.end();
```

Computer Drives & Storage

```
  });
}).listen(3200);
```

Correctly reads the file html/web.html (assuming the folder structure is correct).

1. Sets appropriate status codes (500 for error, 200 for success).
2. Handles errors gracefully.
3. Sends the HTML content as response.

Check Output

1. When you run node web.js or nodemon web.js, and visit http://localhost:3200, the browser will correctly show the HTML content from web.html.

15#########

Form inside JS file

You correctly created and served a basic HTML form directly via resp.write():

```
const http = require("http");
```

Learn Java

```
http
  .createServer((req, resp) => {
    resp.writeHead(200, {
      "Content-Type": "text/html",
    });
    resp.write(`
      <form action="/submit" method="POST">
        <input type="text"  placeholder="Enter username" name="name" />
        <input type="text"  placeholder="Enter email" name="email" />
        <button>Submit</button>
      </form>
    `);
    resp.end();
  })
  .listen(3200);
```

This works fine, though you're not handling the actual POST data yet — just confirming routing works.

Learn Scripting

Conditional Handling of /submit

```
if (req.url === "/") {
  // form page
} else if (req.url === "/submit") {
  // confirmation message
```

}

This conditional routing is correct for handling basic URL-based navigation.

Using a separate form.html file

<!DOCTYPE html>

Build a Website

<html lang="en">

<head>

<meta charset="UTF-8" />

<meta name="viewport" content="width=device-width, initial-scale=1.0" />

<title>Form page</title>

</head>

<body>

<form action="/submit" method="POST">

<input type="text" placeholder="Enter username" name="name" />

<input type="text" placeholder="Enter email" name="email" />

<button>Submit</button>

</form>

</body>

</html

Perfectly structured form with POST method and inputs.

Load HTML File via fs

const http = require('http');

Store Big Data

const fs = require('fs');

```
http.createServer((req, resp) => {
  fs.readFile('html/form.html', 'utf-8', (error, data) => {
    if (error) {
      resp.writeHead(500, { "content-type": 'text/plain' });
      resp.end('internal server error');
      return;
    }

    resp.writeHead(200, { "content-type": 'text/html' });

    if (req.url == '/') {
      resp.write(data);
    } else if (req.url == '/submit') {
      resp.write('<h1>Data submitted</h1>');
    }

    resp.end();
  });
}).listen(3200);
```

Everything is correct:

1. Loads HTML file when GET /
2. Shows confirmation message when /submit is requested
3. Responds with correct headers
4. Properly handles fs.readFile errors

16########

When a form is submitted using the POST method, data is sent in chunks. To handle this in Node.js, we use two important events: `on('data')` and `on('end')`.

hen a form is submitted using the POST method, data is sent in chunks. To handle this in Node.js, we use two important events: on('data') and on('end').

```
const http = require("http");

const fs = require("fs");

const querystring = require("querystring");


http

  .createServer((req, res) => {

    fs.readFile("html/web.html", "utf-8", (err, data) => {

      if (err) {

        res.writeHead(500, { "content-type": "text/plain" });

        res.end("internel srevre error");

        return;

      }

      res.writeHead(200, { "content-type": "text/html" });


      if (req.url == "/") {

        res.write(data);

      } else if (req.url == "/submit") {

        res.write(`<h1>the form has been submitted </h1>`);

        let formData = [];
```

```
      req.on("data", (chunk) => {
        formData.push(chunk);
      });
      req.on("end", () => {
        let fullFormData = Buffer.concat(formData).toString();
        let formatData = querystring.parse(fullFormData);
        console.log(formatData);
      });
    }
    res.end();
  });
})
.listen(7654);
```

Get data from request

We collect incoming data chunks into a buffer:

```
let dataBody = [];
req.on("data", (chunk) => {
  dataBody.push(chunk);
});
```

Handle data with Buffer

Once all data is received, we combine and convert it to a string:

```
req.on("end", () => {
  let rawData = Buffer.concat(dataBody).toString();
  console.log(rawData);
```

});

resp.write(`<h1> Data Submitted </h1>`);

Learn Scripting

Make request data readable

To parse this string into key-value pairs, use Node's built-in querystring module:

```
const querystring = require("querystring");


req.on("end", () => {
  let rawData = Buffer.concat(dataBody).toString();
  let readableData = querystring.parse(rawData);
  console.log(readableData);
});
```

Notes

1. on('data') collects the chunks
2. on('end') finalizes the data
3. querystring.parse() makes the data easier to use

17#########

Synchronous File Creation

```
let dataString =
  "My name is " +
  readableData.name +
```

```
  " and my email is " +
  readableData.email;

fs.writeFileSync("text/" + readableData.name + ".txt", dataString);
console.log("File Created");
```

1. dataString: combines name and email from the form.
2. writeFileSync: writes data to file in a blocking way (waits till complete).
3. "text/" + readableData.name + ".txt": file name is based on user input.

Asynchronous File Creation

```
fs.writeFile(
  "text/" + readableData.name + ".txt",
  dataString,
  "utf-8",
  (err) => {
    if (err) {
      resp.end("Internal Server Error");
      return false;
    } else {
      console.log("File Created");
    }
  }
);
```

1. writeFile: non-blocking (code runs while file is being created).
2. "utf-8": encoding type.
3. Callback handles success or error.

18#########

Main Function Execution

When Node.js runs code, it first registers the **main()** function in the **call stack**. Then other functions are called in order.

```
const x = 1;

const y = x + 2;

console.log("sum is " + y);
```

1. main() is added to the call stack automatically.
2. After console.log(), it exits the stack.
3. Only main() remains and exits last.

Example with a Function

```
const listLocations = (locations) => {
  locations.forEach((location) => {
```

Earn a CS Degree

```
    console.log(location);
  });
};


const myLocations = ["philly", "nyc"];


listLocations(myLocations);
```

Call Stack Flow:

1. listLocations enters call stack

2. forEach runs each item
3. console.log("philly")
4. Stack clears, then next item:
5. console.log("nyc")
6. Stack becomes empty

Learn to Code

Node APIs & setTimeout

Node APIs handle asynchronous code like setTimeout.

```
console.log("Starting up");

setTimeout(() => {
  console.log("Two Seconds");
}, 2000);

setTimeout(() => {
  console.log("Zero Seconds");
}, 0);

console.log("Finishing up");
```

Learn Scripting

Call Stack Output Order:

1. Starting up
2. Finishing up
3. (then Node APIs hold timers)

Event Loop then executes callbacks when the call stack is empty:

3. Zero Seconds

4. Two Seconds

Event Loop

1. Keeps checking if the **call stack** is empty.
2. If empty, moves code from **callback queue** to **call stack**.

19#########

Node.js is great for creating modular server-side applications. In this post, we'll create a simple project using **modules**, **routing**, and **form handling**.

Project Structure

Create 3 files:

1. root.js (main server file)
2. userForm.js (HTML form)
3. userDataSubmit.js (handle form data)

root.js (Main Server File)

```
const http = require("http");

const userForm = require("./userForm");

const userDataSubmit = require("./userDataSubmit");
```

http

```
  .createServer((req, resp) => {
    resp.writeHead(200, { "Content-Type": "text/html" });

    if (req.url === "/") {
      userForm(req, resp);
    } else if (req.url === "/submit") {
      userDataSubmit(req, resp);
    }

    resp.end();
  })
  .listen(3200);
```

This file:

1. Creates the HTTP server
2. Routes / to form and /submit to form handler
3. Sends HTML response based on the route

userForm.js (HTML Form)

```
function userForm(req, resp) {
  resp.write(`
    <form action="/submit" method="POST">
      <input type="text" placeholder="Enter username" name="name" />
      <input type="text" placeholder="Enter email" name="email" />
      <button>Submit</button>
    </form>
```

```
  `);
}

module.exports = userForm;
```

This file:

1. Displays a form with two input fields
2. Sends data to /submit via POST method

userDataSubmit.js (Handle Form Data)

```
const querystring = require("querystring");

function userDataSubmit(req, resp) {
  let dataBody = [];

  req.on("data", (chunk) => {
    dataBody.push(chunk);
  });

  req.on("end", () => {
    let rawData = Buffer.concat(dataBody).toString();
    let readableData = querystring.parse(rawData);

    let dataString =
      "My name is " +
      readableData.name +
      " and my email is " +
```

```
    readableData.email;


  console.log(dataString);
 });


 resp.write(`<h1>You can get data from user form here</h1>`);
}


module.exports = userDataSubmit;
```

This file:

1. Listens to data from the form
2. Collects data chunks using Buffer
3. Converts it to a readable object using querystring
4. Logs name and email in the console

Output Demo

node root.js

Open browser at:

http://localhost:3200/

19#########

n most programming languages, operations run **synchronously** — one task at a time. But in JavaScript and Node.js, we often use **asynchronous programming** for better speed and performance.

Synchronous Programming

1. Executes **one task at a time**.
2. The next line waits until the current one finishes.
3. **Slower** in terms of overall performance.

```
console.log("Apple1");

console.log("Apple2");

console.log("Apple3");
```

Each task waits for the previous one to complete.

Asynchronous Programming

1. **Doesn't wait** for a task to finish before moving on.
2. Great for **I/O operations** (file, API, database).
3. **Faster** in real-world applications.

```
console.log("Apple1");


setTimeout(() => {
  console.log("Apple2");
}, 2000);


console.log("Apple3");
```

setTimeout() runs separately and doesn't block the next line.

When Not to Use Asynchronous Code?

Sometimes, we want code to **wait** until one step is complete.

```
let a = 20;

let b = 0;


setTimeout(() => {
  b = 100;
}, 2000);


console.log(a + b); // Output: 20 (Not 120!)
```

Here, we **don't need** async because we want b before using it.

Real-Life Async Example

Reading a file without blocking the main thread:

```
onst fs = require("fs");


fs.readFile("text/peter.txt", "utf-8", (err, data) => {
  if (err) return false;
  console.log(data);
});


console.log("End Script");
```

Read FIle in Sync

```
const data = fs.readFileSync("text/peter.txt", "utf-8");
```

console.log(data);

20#########

Learn how to perform **Create, Read, Update, and Delete** operations using Node.js's built-in fs module. We'll also handle terminal inputs to do these operations dynamically.

Basic File Operations

const fs = require("fs");

Create New Files

fs.writeFileSync("files/apple.txt", "This is fruit");

fs.writeFileSync("files/banana.txt", "This is fruit");

writeFileSync() creates files. If file already exists, it will overwrite.

Delete a File

fs.unlinkSync("files/banana.txt");

unlinkSync() removes a file.

Read a File

const data = fs.readFileSync("files/apple.txt", "utf-8");

console.log(data);

readFileSync() reads file content. "utf-8" ensures readable string output.

Update a File

fs.appendFileSync("files/apple.txt", " and this is good for health");

appendFileSync() adds new content to existing file.

CRUD Operations with Terminal Input

Now let's make this dynamic — use terminal commands like Write, Read, update, or delete.

```
const fs = require("fs");
const operation = process.argv[2];
```

process.argv[] gives command-line input.

Create File from Terminal

node index.js Write fileName "your content"

```
if (operation === "Write") {
  const name = process.argv[3];
  const content = process.argv[4];
  fs.writeFileSync(`files/${name}.txt`, content);
  console.log("File Created");
}
```

Read File from Terminal

node index.js Read fileName

```
else if (operation === "Read") {
  const name = process.argv[3];
```

```
  const data = fs.readFileSync(`files/${name}.txt`, "utf-8");
  console.log(data);
}
```

Update File from Terminal

node index.js update fileName "new content"

```
else if (operation === "update") {
  const name = process.argv[3];
  const content = process.argv[4];
  fs.appendFileSync(`files/${name}.txt`, content);
}
```

Delete File from Terminal

node index.js delete fileName

```
else if (operation === "delete") {
  const name = process.argv[3];
  fs.unlinkSync(`files/${name}.txt`);
}
```

Fallback Case

```
else {
  console.log("Operation not found");
}
```

Example Test

node index.js Write hello "This is demo"

node index.js Read hello

node index.js update hello " updated!"

node index.js delete hello

21#########

Create a basic website with **HTML and CSS** using **Node.js**. We'll load different HTML pages and a CSS file using the fs (File System) module and http core module.

Browse Reference

Create a server file

website.js

```
const http = require("http");


http
  .createServer((req, resp) => {
    resp.write("page check");
    resp.end();
  })
```

```
  .listen(3200);
```

This creates a simple HTTP server.

Compare Broadband

Load HTML file using fs

```
const http = require("http");
const fs = require("fs");

http
  .createServer((req, resp) => {
    fs.readFile("html/home.html", "utf-8", (error, data) => {
      if (error) {
        resp.writeHead(500, { "content-type": "text/plain" });
        resp.end("Internal Server Error");
        return false;
      }

      resp.write(data);
      resp.end();
    });
  })
  .listen(3200);
```

This loads the home.html file and displays its content on the browser using **asynchronous** fs.readFile().

Sync vs Async in fs (Used in Header)

In the code below, we use:

let collectHeaderData = fs.readFileSync("html/header.html", 'utf-8');

Synchronous (Sync)

Learn to Code

fs.readFileSync() blocks further execution until the file is fully read.

Use it for small files like header.html where immediate availability is needed.

Later in the same code:

fs.readFile("html"+file+".html", 'utf-8', (error, data) => {

  ...

});

Asynchronous (Async)

fs.readFile() does **not block** the code. It continues and runs the callback when data is ready.

Learn Java

Better for performance, especially for larger or external files.

Handle HTML and CSS File Together

```
const http = require('http');
const fs = require('fs');

http.createServer((req, resp) => {

   //  Sync: Header file loaded before anything else
   let collectHeaderData = fs.readFileSync("html/header.html", 'utf-8');
```

```
let file = "/home";
if (req.url !== '/') {
    file = req.url;
}

// Async: Load HTML file with callback
if (req.url !== '/style.css') {
    fs.readFile("html" + file + ".html", 'utf-8', (error, data) => {
        if (error) {
            resp.writeHead(500, { "content-type": "text/plain" });
            resp.end("internal server error");
            return false;
        }

        resp.write(collectHeaderData + "" + data);
        resp.end();
    });
}
// Serve CSS with correct content-type
else if (req.url === '/style.css') {
    fs.readFile("html/style.css", 'utf-8', (error, data) => {
        if (error) {
            resp.writeHead(500, { "content-type": "text/plain" });
            resp.end("css not found");
            return false;
```

```
        }


        resp.writeHead(200, { "content-type": "text/css" });
        resp.end(data);
    });
  }

}).listen(3200);
```

Why CSS Didn’t Load Earlier?

Because we didn’t set "Content-Type": "text/css".

Browsers need the correct header to apply styles.

22#########

What is Path module?

In Node.js, the **path module** is a built-in core module that provides utilities for working with file and directory paths.

What is Global constant?

In Node.js, a **global constant** is a value that is always available in your code without requiring an import or declaration.

Example of both:

```
const path = require("path");
```

```
const file = "text/peter.txt";


console.log(path.extname(file)); // Output: .txt

console.log(path.dirname(file)); // Output: text

console.log(path.basename(file)); // Output: peter.txt

console.log(path.resolve("text", "peter.text")); // Output: absolute path to text/peter.text

console.log(path.isAbsolute(file)); // Output: false (if the path is not absolute)
```

This are the global constants in Node.js

```
console.log(__dirname); // Output: current directory path

console.log(__filename); // Output: current file path
```

Why do we need global constants?

This is very helpful when you're working with file systems (like reading HTML or CSS files), especially when your project is running on different machines or environments. It ensures your file paths are always accurate.

EXPRESS JS

23#########

What is Express.js?

Express.js is a fast, minimal, and flexible web application framework built on **Node.js**. It helps you easily create web servers, APIs, and routes.

Learn Scripting

Why Use Express.js?

1. Simple Routing
2. Middleware Support
3. Template Engines
4. REST API Ready
5. Lightweight and Fast
6. Easy to Learn

How to Set Up Express.js

Create a New Project Folder

Express-2026

Initialize Node.js Project

npm init

Try Dev Tools

Fill the details like:

1. Package Name: node-js-2026
2. Description: node js and express js tutorials in hindi 2026
3. Entry Point: index.js (default)

Install Express

npm install express --save

Create Entry File – index.js

Basic Express Server

```
const express = require("express");
```

Learn Java

```
const app = express();

app.get("", (req, resp) => {
  resp.send("<h1>Basic Node js example</h1>");
});

app.get("/about", (req, resp) => {
  resp.send("<h1>This is about page</h1>");
});

app.get("/contact", (req, resp) => {
  resp.send("<h1>This is contact page</h1>");
});

app.listen(3200);
```

Explanation:

1. app.get() – Handles GET requests.
2. resp.send() – Sends HTML or plain text response.
3. app.listen(3200) – Server listens on port 3200.

Learn to Code

Auto Restart with Nodemon

To avoid restarting the server manually every time:

Install Nodemon

npm i nodemon --save-dev

Then run your project using:

npx nodemon index.js

Now your server will auto-refresh on every file save.

24#########

Understand Express.js Flow

Require Express

const express = require("express");

We use require("express") to include the Express module.

Why we can't use Express directly?

Because Express returns a function. To use it, we need to call it:

const app = express();

Alternatively, you can do it like this in one line:

const app = require("express")();

This creates an Express application and stores it in the app variable.

Start the Server

app.listen(3200);

This tells Express to start a server on port 3200.

Check app Object

If you want to inspect what's inside the Express app, use:

console.log(app);

This will show all the available methods and properties of the Express app.

Express.js Routes and Flow

Basic Route Example

```
app.get("", (req, resp) => {
  resp.send("<h1>Home Page</h1>");
});
```

1. .get() is used to handle **GET** requests.
2. It takes **two parameters**: a path ("" or "/about") and a callback function.

Multiple Routes Example

```
app.get("", (req, resp) => {
  resp.send("<h1>Home Page</h1>");
});


app.get("/about", (req, resp) => {
  resp.send("<h1>About Page</h1>");
});
```

```
app.get("/", (req, resp) => {
  resp.send("<h1>Home Page 2</h1>");
});
```

Note:

Express matches and executes the **first matching route**. So if "" and "/" look similar, only the first one is executed.

25#########

Why ES Modules?

By default, Node.js uses **CommonJS** syntax (require, module.exports).

If you try:

Learn Java

```
import express from 'express';
```

You'll get an error, because Node doesn’t recognize import/export without proper setup.

Enable ES Module Support

In your package.json, add this line:

```
"type": "module"
```

This tells Node.js to interpret your files using **ES module syntax**.

Before (CommonJS Syntax)

```
const express = require("express")();
```

After (ES (ECMAScript) Module Syntax)

```
import express from 'express';
```

```
const app = express();
```

Create Pages Using Export/Import

Create home.js

pages/home.js

```
export default function home() {
  return `<h1>Home Page with ES export way</h1>`;
}
```

Create about.js

File: pages/about.js

```
export default function about() {
  return `<h1>About Page</h1>`;
}
```

Create contact.js

pages/contact.js

```
export default function contact() {
  return `<h1>Contact Page</h1>`;
}
```

Import and Use in Your Main File

File: index.js

```
import express from 'express';
import home from './pages/home.js';
import about from './pages/about.js';
```

```
import contact from './pages/contact.js';

const app = express();

app.get("/", (req, res) => {
  res.send(home());
});

app.get("/about", (req, res) => {
  res.send(about());
});

app.get("/contact", (req, res) => {
  res.send(contact());
});

app.listen(3200);
```

27#########

1. Render basic HTML elements
2. Create and display a login form
3. Handle POST requests
4. Import HTML from other files and serve them using Express.js

Setup Express

Start by importing express and creating an app instance:

```
import express from "express";

const app = express();
```

Render Basic Element on Home Page

```
app.get("/", (req, resp) => {
  resp.send("<h1>Home Page</h1>");
});
```

This will render a simple **Home Page** when you visit http://localhost:3200/.

Make and Render a Login Form

```
app.get("/login", (req, resp) => {
  resp.send(`<form action="/submit" method="post">
                        <input type="text" placeholder="name">
                        <br/>
                        <br/>
                        <input type="password" placeholder="Enter Password">
                        <br/>
                        <br/>
                        <button>Login</button>
                        </form>`);
});
```

Visit http://localhost:3200/login to view a basic login form.

Handle POST Request (Form Submission)

```
app.post("/submit", (req, resp) => {
  resp.send(`<h1>Data Sumitted</h1>`);
});
```

When the form is submitted, this route handles the POST request and returns a success message.

Import HTML Elements From Other Files

Organize our HTML into separate files and import them.

home.js

```
export default function home() {
  return "<h1>Home Page</h1> <a href='/login'>Go to Login</a>";
}
```

login.js

```
export default function login() {
  return `<form action="/submit" method="post">
                        <input type="text" placeholder="name">
                        <br/>
                        <br/>
                        <input type="password" placeholder="Enter Password">
                        <br/>
                        <br/>
                        <button>Login</button>
                        <a href='/'>Go to Home</a>
                        </form>`;
}
```

submit.js

```
export default function submit() {
  return "<h1>Data Sumitted</h1> <a href='/'>Go to Home</a>";
}
```

Final Code — index.js

```
import home from "./pages/home.js";
import login from "./pages/login.js";
import submit from "./pages/submit.js";

const app = express();

app.get("/", (req, resp) => {
  resp.send(home());
});
app.get("/login", (req, resp) => {
  resp.send(login());
});

app.post("/submit", (req, resp) => {
  resp.send(submit());
});

app.listen(3200, () => {
  console.log("Server is running on port 3200");
```

});

1. Visit / to see the home page.
2. Visit /login for the login form.
3. Submit the form and it will show the confirmation from /submit.

28#########

Create and render HTML files

home.html

```
<h1>Home Page</h1>
<a href="login">Go to Login Page</a>
```

login.html

```
<h1>Login Page</h1>
<a href="/">Go to Home Page</a>
```

about.html

```
<h1>About Page</h1>
<a href="/">Go to Home Page</a>
```

Make node server with express js

```
import express from "express";

const app = express();

app.get("/", (req, resp) => {
  resp.sendFile("view/home.html");
```

```
});
app.listen(3200);
```

```
resp.sendFile("view/home.html");
```

This gives you an error because you need an **absolute path**.

Get Absolute Path

Use the path module to get absolute path.

```
import express from "express";
import path from "path";

const app = express();

app.get("/", (req, resp) => {
  const absPath = path.resolve("view/home.html");
  resp.sendFile(absPath);
});

app.get("/login", (req, resp) => {
  const absPath = path.resolve("view/login.html");
  resp.sendFile(absPath);
});

app.get("/about", (req, resp) => {
  const absPath = path.resolve("view/about.html");
```

```
  resp.sendFile(absPath);
});
app.listen(3200);
```

1. We used path.resolve() to get the absolute path of HTML files.

Now you're ready to open the browser:

1. Home → http://localhost:3200/
2. Login → http://localhost:3200/login
3. About → http://localhost:3200/about

29#########

What is a 404 Page?

A **404 page** means **"Page Not Found"**. This shows up when a user tries to visit a page on your website that doesn't actually exist.

Why is 404 Page Important?

Without a 404 page:

1. User sees a blank or error page.
2. They may think your website is broken or not working.

With a 404 page:

1. You can show a friendly message.
2. Tell the user that the website is fine, but the page they requested doesn't exist.

Make a File for 404 Page

Create a new HTML file named 404.html inside the view folder.

You can use ChatGPT to generate a beautiful and simple 404 HTML template.

Use Function for 404 Page in Express

Express provides a special function app.use() — it's a **multi-purpose middleware** and is also used to handle 404 errors.

```
app.use((req, resp) => {
  const absPath = path.resolve("view/404.html");
  resp.status(404).sendFile(absPath);
});
```

Improvement: Make Common Absolute Path

To avoid repeating path.resolve() again and again, create a **common absolute path** for the whole folder (not individual files):'

```
const absPath = path.resolve("view");
```

Then use it like this:

```
app.use((req, resp) => {
  resp.status(404).sendFile(absPath + "/404.html");
});
```

######

Add CSS in HTML Page (Inline Method)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Home Page</title>
```

```
  <style>
    h1 {
      color: orange;
    }

    a {
      text-decoration: none;
      padding: 10px 20px;
      background: blue;
      color: white;
      border-radius: 10px;
    }

    a:hover {
      background: rgb(86, 86, 220);
    }
  </style>
 </head>
 <body>
  <h1>Home Page</h1>
  <a href="login">Go to Login Page</a>
 </body>
</html>
```

Is file mein <style> tag ke andar hi CSS likha gaya hai. Yeh basic styling test karne ke liye kaafi hai.

Make External CSS File

Create a file:

public/css/style.css

style.css

```
h1 {
  color: orange;
}

a {
  text-decoration: none;
  padding: 10px 20px;
  background: blue;
  color: white;
  border-radius: 10px;
}

a:hover {
  background: rgb(86, 86, 220);
}
```

Add CSS Link in HTML

In home.html file head section, change:

```
<link rel="stylesheet" href="css/style.css" />
```

✕ Don't use:

```
<link rel="stylesheet" href="public/css/style.css" />
```

Serve Static Folder in Express

```
import express from "express";

import path from "path";


const app = express();


const publicPath = path.resolve("public");


app.use(express.static(publicPath));
```

The express.static() function allows you to make the files inside the public folder accessible to the browser.

So when your HTML file includes a line like:

```
<link rel="stylesheet" href="css/style.css" />
```

…the browser will be able to **find and load** that CSS file from the public/css folder, **because** you've told Express to serve everything in the public folder using:

```
app.use(express.static(publicPath));
```

Without this, the browser wouldn’t be able to locate the CSS file, even if it's in your project folder. This function tells Express:

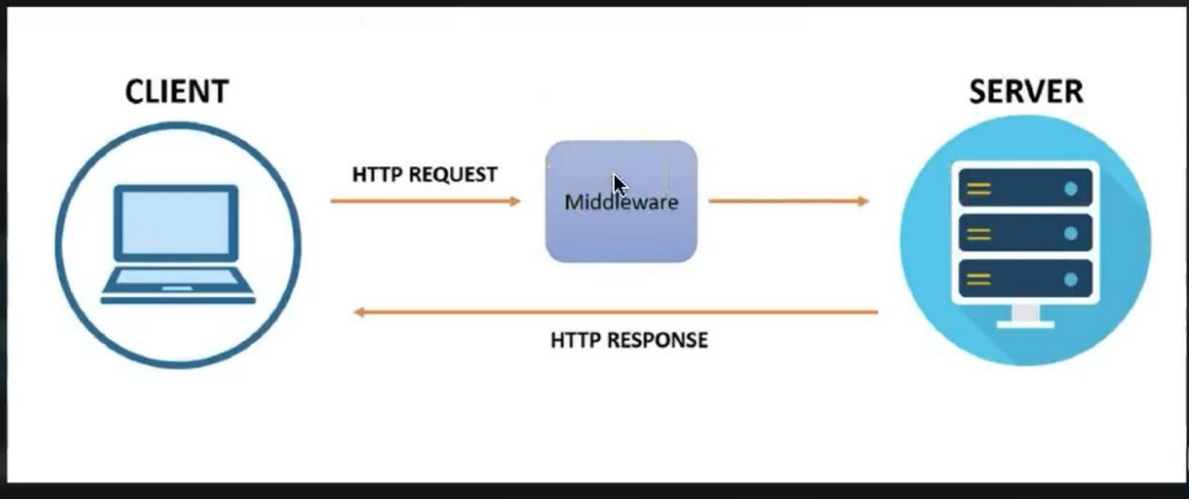
30#########

Middleware in express js

Middleware in Express.js is a **function**
-that runs **before the final route handler**

```
1 function middleware(req, res, next) {
2   // Do something with req or res
3   next(); // Pass control to the next middleware or route
4 }
5
6
7
8 app.use(authMiddleware); // Applies to all routes
```

Middleware in express js



What is Middleware in Express.js?

Middleware in Express.js is a **function** that gets executed **before** the final route handler.

It can be used to check requests, log activities, authenticate users, validate data, and more.

How Middleware Works

```
function middleware(req, res, next) {
  // Do something with req or res
  next(); // Pass control to the next middleware or route
}
```

1. req is the request object
2. res is the response object
3. next() moves to the next middleware or route handler

How the Client, Middleware, and Server Work Together

Client → Sends HTTP request to server (e.g., GET, POST)

Middleware → Intercepts and processes the request

Server → Executes the final route handler and sends a response

Middleware Example (code)

```
import express from "express";

const app = express();

function checkRoute(req, resp, next) {
  console.log("user is acessing " + req.url + " Page");
```

```
  next();
}

app.use(checkRoute);

app.get("/", (req, resp) => {
  resp.send("Home Page");
});

app.get("/users", (req, resp) => {
  resp.send("users Page");
});

app.get("/products", (req, resp) => {
  resp.send("Products Page");
});

app.listen(3200);
```

This middleware (checkRoute) runs for **every route** and logs the page being accessed before the actual route is handled.

You can also write it like this using an arrow function:

```
app.use((req, resp, next) => {
  console.log("user is acessing " + req.url + " Page");
  next();
});
```

Types of Middleware

Application-level Middleware

1. Registered using: app.use() or app.get()
2. Purpose: Used to apply middleware globally or to specific routes in the main app.
3. Example Use: Logging, request parsing, etc.

Router-level Middleware

1. Registered using: router.use()
2. Purpose: Works like application-level middleware, but applies to route instances (e.g., for grouping /admin or /user routes).
3. Example Use: Authorization checks in a specific route group.

Built-in Middleware

1. Comes with Express.js.
2. Common example: express.static() to serve static files (like CSS, images).
3. Purpose: Provides ready-to-use functionality without extra installation.

Error-handling Middleware

1. Defined with four arguments: (err, req, res, next)
2. Purpose: Catches and handles errors across the application.
3. Example Use: Sending custom error messages, logging errors.

Third-party Middleware

1. Installed via npm, and used with require() or import.
2. Purpose: Add advanced functionalities.
3. Common examples:
4. morgan (logging)
5. cors (Cross-Origin Resource Sharing)
6. body-parser (parsing form data)

Benefit Middleware in express js

- Modifies Request
- Centralized Logic
- Auth
- Third-party Support
- Static file handling
- 404 page handling

Types of Middleware in express js

Type	Registered With	Purpose
Application-level	`app.use()` / `app.get()`	Global or specific route handling
Router-level	`router.use()`	Modular route handling
Built-in	`express.*`	Parsing, static serving, etc.
Error-handling	`(err, req, res, next)`	Handle app errors
Third-party	via `require()`	Logging, CORS, authentication, etc.

Create Server

```
import express from "express";

const app = express();

app.get("/", (req, resp) => {
  resp.send("<h1>Home Page</h1>");
});

app.get("/login", (req, resp) => {
  resp.send("<h1>Login Page</h1>");
});

app.get("/admin", (req, resp) => {
  resp.send("<h1>Admin Page</h1>");
});

app.listen(3200);
```

Middleware for Age Check

Function Purpose:

1. Checks if user provided age in the x
2. Blocks access if age is missing or less than 18

Code:

```
function ageCheck(req, resp, next) {
  if (!req.query.age || req.query.age < 18) {
    resp.send("Alert ! You can not access this page");
  } else {
    next();
  }
}

app.use(ageCheck); // Applies globally to all routes
```

Middleware for IP Check

Function Purpose:

1. Logs the visitor's IP address
2. Blocks access for specific IP (example: 192.168.1.23)

For windows

```
function ipCheck(req, resp, next) {
  const ip = req.ip;
  console.log(ip); // View in terminal

  if (ip ==="::ffff:192.168.1.23" || ip ==="192.168.1.23"   )) {
    resp.send("Alert ! You can not access this page");
  } else {
    next();
  }
```

```
}
```

For mac os

```
function ipCheck(req, resp, next) {
  const ip = req.socket.remoteAddress;
  console.log(ip); // View in terminal

  if (ip.includes("192.168.1.23")) {
    resp.send("Alert ! You can not access this page");
  } else {
    next();
  }
}

app.use(ipCheck); // Applies globally
```

How to Find Your IP Address

Windows Command Prompt:

ipconfig

Mac Terminal:

ipconfig getifaddr en0

31#########

What is Route Middleware?

1. Middleware applied to **specific routes only**
2. Executes **before the final route handler**

How to Make Route Middleware

```
function checkAgeRouteMiddleware(req, resp, next) {
  console.log(req.query);

  if (!req.query.age || req.query.age < 18) {
    resp.send("Yoru are not allowed to use this website");
  } else {
    next();
  }
}

function checkUrl(req, resp, next) {
  console.log("this req url is " + req.url);
  next();
}
```

How to Apply Route Middleware

1. Use middleware as a **second parameter**
2. You can apply **multiple middleware functions** on a single route

```
app.get("/", (req, resp) => {
  resp.send("<h1>Home page</h1>");
});

app.get("/login", checkUrl, (req, resp) => {
  resp.send("<h1>login page</h1>");
});

app.get("/users", checkAgeRouteMiddleware, checkUrl, (req, resp) => {
  resp.send("<h1>users page</h1>");
});

app.get("/products", checkAgeRouteMiddleware, checkUrl, (req, resp) => {
  resp.send("<h1>products page</h1>");
});
```

30#########

Built-in middleware is already provided by Express. You **don't need to create** it — you just **use it directly**.

Common Built-In Middleware

express.static()

➤ Serves static files like CSS, JS, images.

➤ Example: Serves files from a folder (public).

expsress.urlencoded()

➤ Parses incoming form data (application/x-www-form-urlencoded).

➤ Helps you access form data via req.body.

Basic Code

```
import express from "express";

const app = express();
```

Form Data Undefined

```
app.post("/submit", (req, resp) => {
  console.log("user login details are :", req.body); // undefined without middleware
  resp.send("Submit Page");
});
```

Use built-in middleware before routes:

```
app.use(express.urlencoded({ extended: false }));
```

This allows Express to read data from the form (like email & password).

Use express.static()

To serve static CSS file:

1. **Create folder**: public
2. **Add file**: public/style.css
3. **In HTML**:

<link rel="stylesheet" href="/style.css" />

In index.js:

app.use(express.static("public"));

Now Express will automatically serve CSS and other static files from public/.

Folder Structure:

project/

├── public/

| └── style.css

├── view/

| └── home.html

└── server.js

31#########

What is External Middleware in Express.js?

1. External middleware is **not built into Express.js**.
2. You must **install** it separately via npm.
3. It adds **extra functionality** (like logging, CORS, authentication, etc.)

Example: morgan Middleware

1. morgan is an external middleware used for **logging HTTP requests** in the terminal.
2. Helpful for **debugging** and **monitoring** incoming requests.

Steps to Use Morgan Middleware

1. Install morgan

npm i morgan

Import it

import morgan from "morgan";

Use it in your Express app

app.use(morgan("dev"));

"dev" is a predefined format that logs method, URL, status code, response time, etc.

Example Output in Terminal

GET / 200 2.012 ms - 9

GET /login 200 1.223 ms - 12

GET /random 404 3.314 ms - 142

To Verify Time Logging

Add a route with a **delay**:

```
app.get("/wait", (req, resp) => {
  setTimeout(() => {
    resp.send("result after 1 sec");
  }, 1000);
```

});

Now open:

http://localhost:3200/wait

Output in Terminal:

GET /wait 200 1011.997 ms - 18

Shows accurate request timing.

Format	Kaam
dev	Development ke liye short, colored logs
tiny	Bohat short logs
short	Short logs, response time ke saath
common	Standard Apache-style logs
combined	Detailed logs, jaise IP aur user-agent

`

32#########

What is Error-Handling Middleware?

1. It is a **special type of middleware** in Express.js.
2. It catches and handles **errors** that occur during route processing.
3. Helps to show a **friendly message** instead of crashing the server.

Basic Error Handler

```
app.get("/users", (req, resp) => {
  resp.send1("User Page"); // Typo will cause error
});
```

```
function errorHandling(error, req, resp, next) {
  resp.status(error.status || 500).send("Try after some time");
}
app.use(errorHandling);
```

Output: Server doesn't crash, shows:

"Try after some time"

Custom Error with next(error)

```
app.get("/error", (req, resp, next) => {
  const error = new Error("Something went wrong");
  error.status = 404;
  next(error); // Pass error to middleware
});
```

```
app.use((error, req, resp, next) => {
  resp.status(error.status || 500).send("Try after some time");
});
```

Output: Returns status 404 with message

“Try after some time”

33########

What is a Template Engine?

1. A **template engine** lets you **generate dynamic HTML** using **server-side data**.
2. Helps merge backend data with frontend templates (like variables in HTML).

Popular Template Engines

1. **EJS** (Embedded JavaScript) ✅
2. **Pug** (formerly Jade)
3. **Handlebars**

How to Install EJS

npm i ejs

EJS File Structure

project/

```
|
├── views/
|   └── home.ejs
├── public/
|   └── style.css
└── index.js
```

Note: EJS **automatically looks inside the views/ folder**

Example: home.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Home Page</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <h1>Name <%= name %></h1>
  <h2>Yt Channel: <%= ytChannel %></h2>
  <h2>Age : <%= age %></h2>
</body>
</html>
```

1. <%= ... %> is used to embed JavaScript values in HTML

index.js

```
import express from "express";
const app = express();

app.set("view engine", "ejs");

app.get("/", (req, resp) => {
  resp.render("home", {
    name: "Anil Sidhu",
    ytChannel: "Code Step By Step",
    age: 29,
  });
});

app.listen(3200);
```

app.set("view engine", "ejs"); tells Express to use EJS

resp.render("home", {...}) sends data to the EJS file

Output at http://localhost:3200/:

Name Anil Sidhu

Yt Channel: Code Step By Step

Age : 29

34#########

Make form with ejs template engine

Create files in view

addUser.ejs

```
<form action="/submit-user" method="post">
  <input type="text" placeholder="enter user name" name="name" />
  <br /><br />
  <input type="text" placeholder="enter user age" name="age" />
  <br /><br />
  <input type="text" placeholder="enter user email" name="email" />
  <br /><br />
  <button>Add User</button>
</form>
```

This form sends data using POST method to the route /submit-user.

submitUser.ejs

```
<h3>Name: <%= name %></h3>
<h3>age:<%= age %></h3>
<h3>Email:<%= email %></h3>
```

Make routes

Now, connect the frontend form and result page using routes in your Express server.

```
app.get("/add-user", (req, resp) => {
  resp.render("addUser");
});

app.post("/submit-user", (req, resp) => {
```

```
  console.log(req.body);
  resp.render("submitUser", req.body);
});
```

app.get loads the form

app.post receives form data and sends it to the template

Get Submitted data

Use req.body to receive the data that the user submitted via the form.

Important: To make this work, we must add this middleware:

```
app.use(express.urlencoded({ extended: false }));
```

Display submitted on ejs template page

In submitUser.ejs, we already used EJS tags to display values:

```
<h3>Name: <%= name %></h3>
<h3>age:<%= age %></h3>
<h3>Email:<%= email %></h3>
```

These values come from the req.body we passed in the POST route.

35#########

Make Route and Pass Array in EJS File

In your index.js file (or main Express file), first set up a route that will render your EJS file.

```
app.get("/users", (req, resp) => {
  resp.render("users"); // This will render 'users.ejs' file
});
```

Note: EJS by default looks inside the views folder, so make sure your EJS file is there.

Create EJS File

Inside the views folder, create a new file called users.ejs.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Users List</title>
  </head>
  <body>
    <h1>User List Page</h1>
  </body>
</html>
```

Create an Array and Pass it to EJS

Now go back to your route and pass a list of users to the template.

```
const users = ["anil", "sidhu", "sam", "peter", "bruce"];


app.get("/users", (req, resp) => {
  resp.render("users", { users: users });
});
```

Here, we are passing the users array to the users.ejs file. You can now use this data inside the template.

Apply Loop in EJS File

Now, inside users.ejs, use a for loop to show each user in a list:

```
<ul>
  <% for (let i = 0; i < users.length; i++) { %>
    <li><%= users[i] %></li>
  <% } %>
</ul>
```

1. Use <% %> to write JavaScript (like loops and conditions).
2. Use <%= %> to print values to HTML.
3. **Don't use =** if you're not printing anything.

Apply Condition in EJS

Let's say we want to show a message only if the user is logged in. First, create a variable in your route:

```
app.get("/users", (req, resp) => {
  const isLogin = true;
  resp.render("users", { users: users, isLogin });
});
```

Now use this variable in your EJS file with an if condition:

```
<% if (isLogin) { %>
  <h2>User is Logged In</h2>
<% } %>
```

MVC ARCHITECTURE

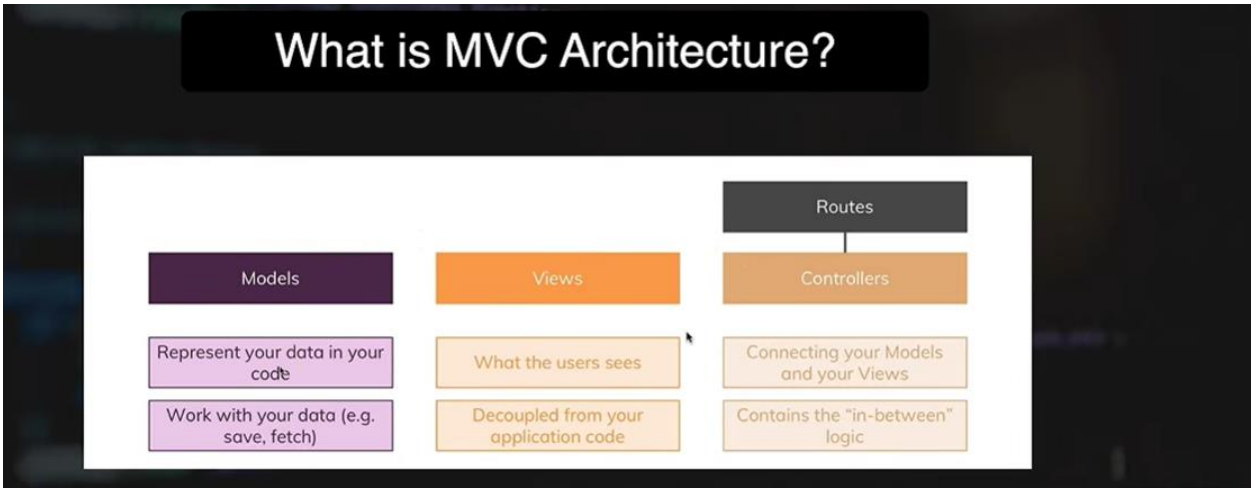
35#########

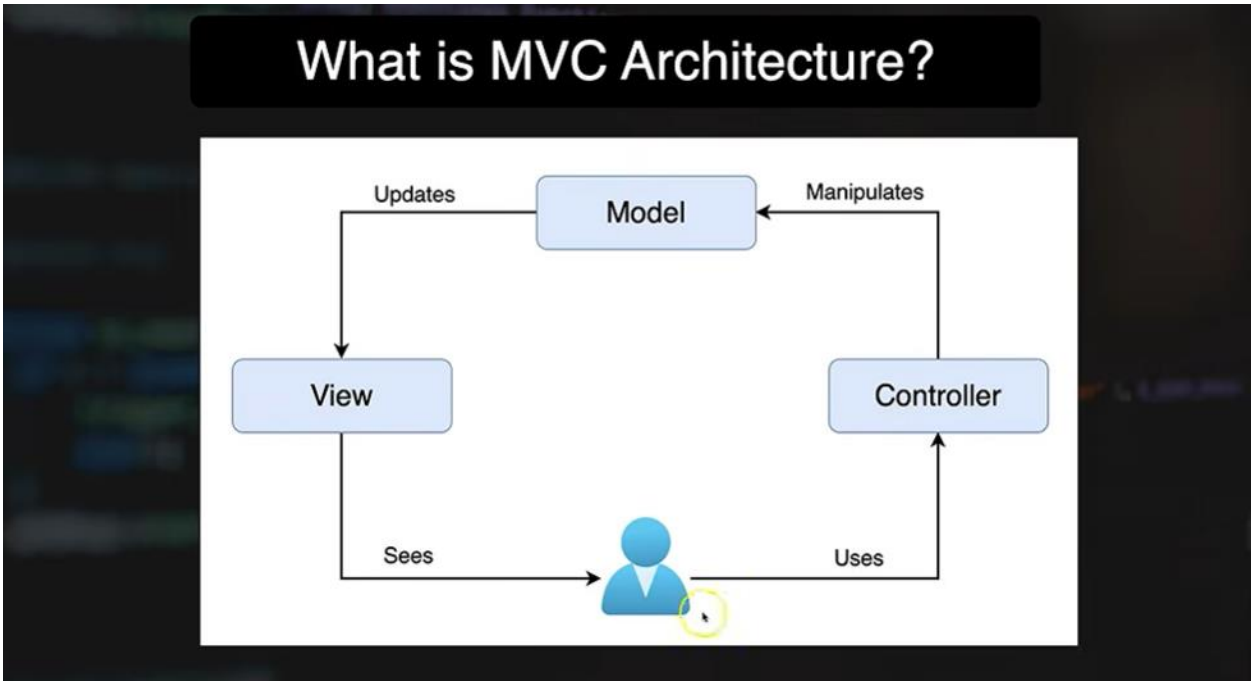
When working on real-world Node.js projects, it's important to organize your code properly. One of the most common and effective patterns for doing this is the **MVC Architecture**.

What is MVC?

MVC stands for:

1. **M** – Model (Handles the data)
2. **V** – View (Handles the UI)
3. **C** – Controller (Handles the logic and routes)





MVC Flow Summary

Here's how everything connects:

User → Controller → Model → View → User

1. The **User** interacts with the app.
2. The **Controller** receives the request and communicates with the **Model**.
3. The **Model** processes data and sends it back.
4. The **View** displays the result to the user.

File Structure in MVC

Create the following folders in your project:

project/

├— controllers/

| └— usersController.js

├— models/

```
|   └── usersModel.js
├── views/
|   └── users.ejs
├── index.js
```

Step-by-Step Setup

Create Basic Express App

In index.js:

```
import express from "express";
import { handleUsers } from "./controllers/usersController.js";

const app = express();
app.set("view engine", "ejs");

// Route
app.get("/user", handleUsers);

app.listen(3200, () => {
  console.log("Server is running on port 3200");
});
```

Create Controller – controllers/usersController.js

The controller connects the model and view.

```
import { userList } from "../models/usersModel.js";

export function handleUsers(req, resp) {
```

```
  const usersData = userList(); // Get data from model
  resp.render("users", { users: usersData }); // Send data to view
}
```

Create Model – models/usersModel.js

```
export function userList() {
  return ["Anil", "Sidhu", "Sam", "Peter", "Bruce"];
}
```

Create View – views/users.ejs

This is what the user sees on the browser.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Users List</title>
  </head>
  <body>
    <h1>User List Page</h1>
    <ul>
      <% for (let i = 0; i < users.length; i++) { %>
        <li><%= users[i] %></li>
      <% } %>
    </ul>
  </body>
```

</html>

We loop through the users array and display each user in a list.

36#########

In real-world applications, we often need to create web pages that respond differently based on the data **passed in the URL**. That's exactly where **dynamic routes** come in!

Store Big Data

What Are Routes?

In a web app, when a user visits a specific **URL**, we use a **route** to handle that request.

Example:

```
app.get("/about", (req, resp) => {
  resp.send("About Page");
});
```

This route handles requests made to localhost:3200/about.

What Are Dynamic Routes?

Dynamic routes allow us to pass dynamic data **through the URL** using parameters.

Upgrade Network

Instead of writing a new route for every user, we write one route that accepts any user name as a **parameter**.

Creating a Dynamic User Profile Page

Create a List of Users with Profile Links

```
app.get("/", (req, resp) => {
  const users = ["anil", "sam", "peter", "sidhu"];
  let data = `<ul>`;

  for (let i = 0; i < users.length; i++) {
    data += `<li><a href="/user/${users[i]}">${users[i]}</a></li>`;
  }

  data += `</ul>`;
  resp.send(data);
});
```

1. This will render a list of user names as links like:

```
/user/anil
/user/sam
```

Create a Dynamic Route to Handle User Profile

```
app.get("/user/:name", (req, resp) => {
  const userName = req.params.name;
  resp.send(`This is ${userName}'s Profile Page`);
});
```

1. Here, :name is a dynamic parameter.
2. req.params.name captures whatever is passed in the URL.

How It Works:

1. Visiting /user/anil will display:
2. **"This is anil's Profile Page"**
3. Visiting /user/sidhu will display:
4. **"This is sidhu's Profile Page"**

This is super useful for applications like:

1. Blog posts (/post/:id)
2. User profiles (/user/:username)
3. Product pages (/product/:slug)

37#########

Create a JSON File with User Data

We'll start by storing our user data in a separate JSON file. Create a folder called data and inside it, create a file named:

user.json

```
[
 {
   "id": 1,
   "name": "Anil",
   "age": 30,
   "email": "anil@test.com"
```

```
  },
  {
    "id": 2,
    "name": "Sam",
    "age": 32,
    "email": "sam@test.com"
  },
  {
    "id": 3,
    "name": "Peter",
    "age": 20,
    "email": "peter@test.com"
  },
  {
    "id": 4,
    "name": "Bruce",
    "age": 18,
    "email": "bruce@test.com"
  }
]
```


Set Up Express Server

Now, create an Express server in your index.js or app.js file:

```
import express from "express";
import userData from "./data/user.json" assert { type: "json" };
```

```
const app = express();
const PORT = 3200;

// Default route to send full user list
app.get("/", (req, resp) => {
  resp.send(userData);
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Fetch a Single User by ID (Dynamic Route)

Create a dynamic route to fetch user data using the ID passed in the URL:

```
app.get("/user/:id", (req, resp) => {
  const id = req.params.id;
  const filteredData = userData.filter((user) => user.id == id);
  resp.send(filteredData);
});
```

Explanation:

1. :id is a dynamic parameter.
2. req.params.id captures the ID from the URL.
3. We use filter() to match the ID in the JSON data.

GET /user/2 → returns data of Sam.

Fetch a User by Name (Case-Insensitive Match)

You can also get users by their name:

```
app.get("/user-by-name/:name", (req, resp) => {
  const name = req.params.name;
  const filteredData = userData.filter(
    (user) => user.name.toLowerCase() === name.toLowerCase()
  );
  resp.send(filteredData);
});
```

Example:

GET /user-by-name/peter → returns Peter's details.

MONGO DB

38#########

What is Database?

1. A database is a structured collection of data
2. That is stored and organized in a way that makes it easy to retrieve, manage, and update.

Types of Database

1. Relational Databases (RDBMS)
2. NoSQL Databases (MongoDB)
3. NewSQL Databases
4. Time Series Databases
5. Object-Oriented Databases
6. Graph Databases
7. Hierarchical Databases

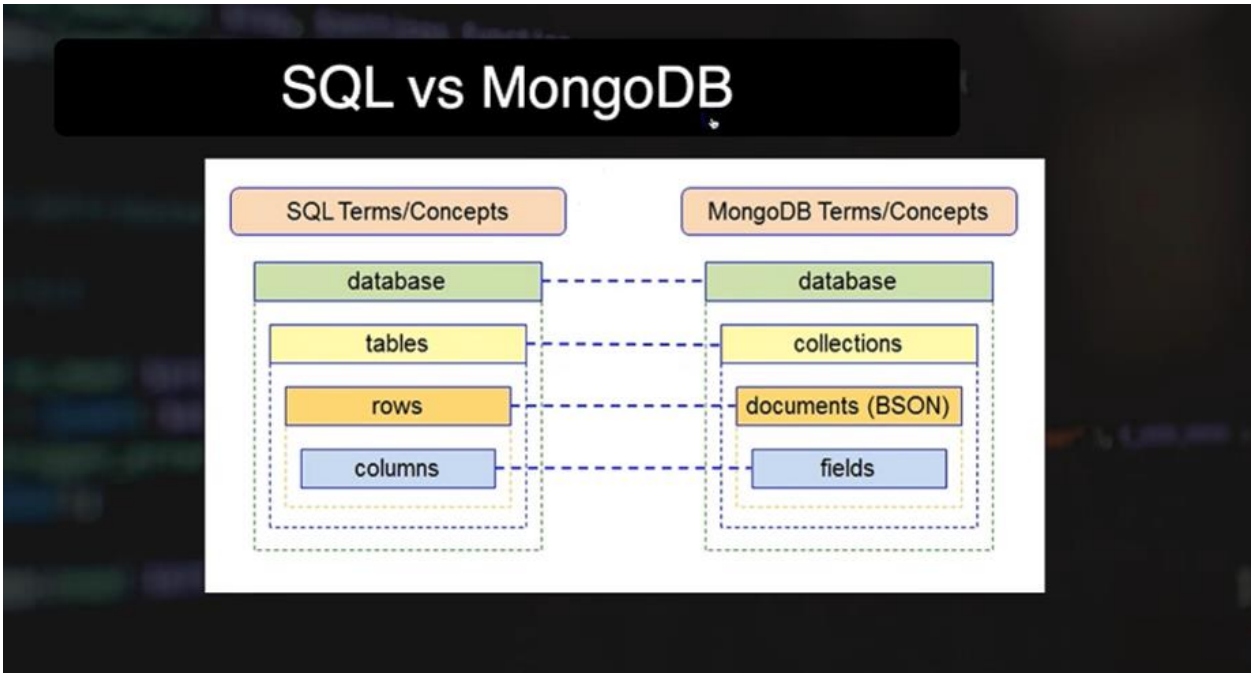
What is MongoDB?

1. MongoDB is a NoSQL database.
2. It stores data in a flexible (no table format)
3. JSON-like format called BSON (Binary JSON).

SQL vs MongoDB

SQL Terms and their MongoDB Equivalents:

1. **SQL: database → MongoDB: database**
2. **SQL: tables → MongoDB: collections**
3. **SQL: rows → MongoDB: documents (BSON)**
4. **SQL: columns → MongoDB: fields**



INSTALLATION MONGO DB

Complete Guide: How to Install MongoDB on Windows 10/11

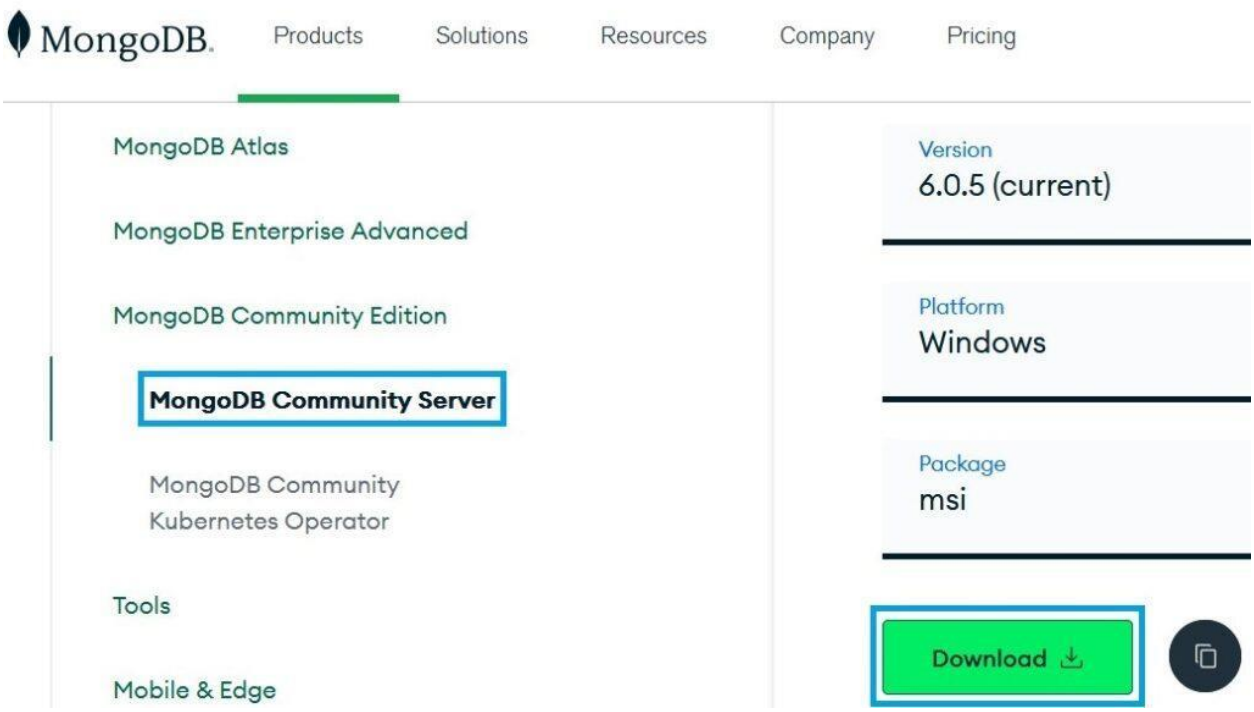
Beginner friendly Windows 10 & 11

In this guide, we'll install MongoDB Community Server on Windows, configure it, connect to it, and run your first database commands.

By the end, you will have:

- MongoDB installed on your Windows computer.
- MongoDB running as a Windows service.
- MongoDB Shell (mongosh) installed.
- MongoDB Compass (GUI) installed.
- A working database and collection.
- Successfully inserted and retrieved data.

Step 1: Download MongoDB



1. Open the official MongoDB Community Server download page:

[Download MongoDB Community Server]

2. Select these options:

Option	Selection
Version	Latest stable version

Platform Windows

Package MSI

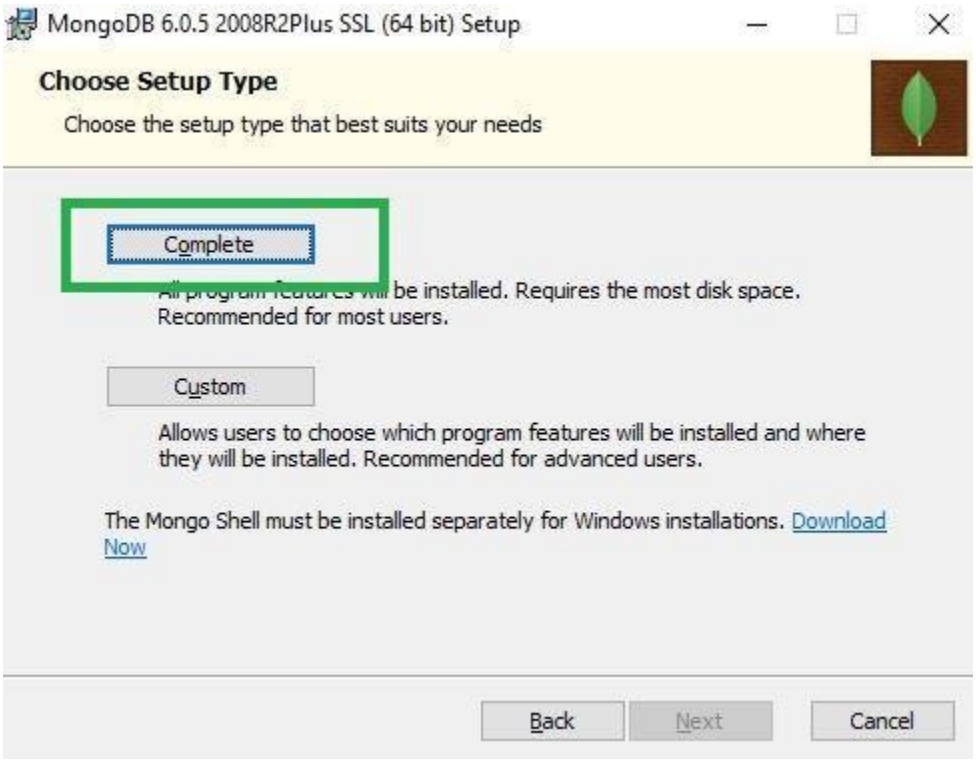
3. Click Download.
4. Wait for the installer to download.

Important: Choose the MSI package, not the ZIP package. MSI makes installation easier for beginners.

Step 2: Install MongoDB



1.
2. Open your downloaded .msi file.
3. Click Next.
4. Accept the license agreement.
5. Click Next.
6. Select Complete installation.



Step 3: Configure MongoDB as a Windows Service

During installation, you will see a Service Configuration screen.

Use these settings:

Setting	**Value**
Install MongoD as a Service	Checked
Run service as	Network Service user
Service Name	MongoDB
Data Directory	C:\Program Files\MongoDB\Server\VERSION\data or the installer's suggested default
Log Directory	Suggested default

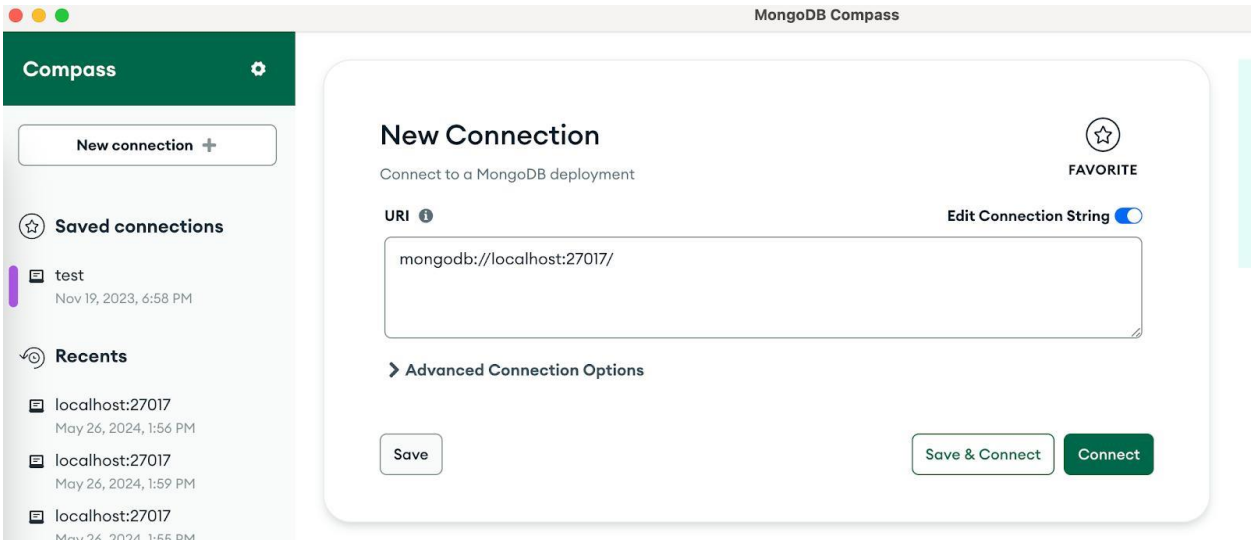
The installer may suggest different data and log paths. You can keep its defaults.

Make sure Install MongoD as a Service is checked.

This allows MongoDB to start automatically when Windows starts.

Click Next.

Step 4: Install MongoDB Compass



MongoDB Compass is a graphical application that lets you manage databases without writing every command manually.

During the MongoDB installation, if you see:

Install MongoDB Compass (Optional)

Keep it checked.

Then:

1. Click Next.
2. Click Install.
3. Wait for installation to finish.
4. Click Finish.

If you did not install Compass during setup, download it here:

[Download MongoDB Compass]

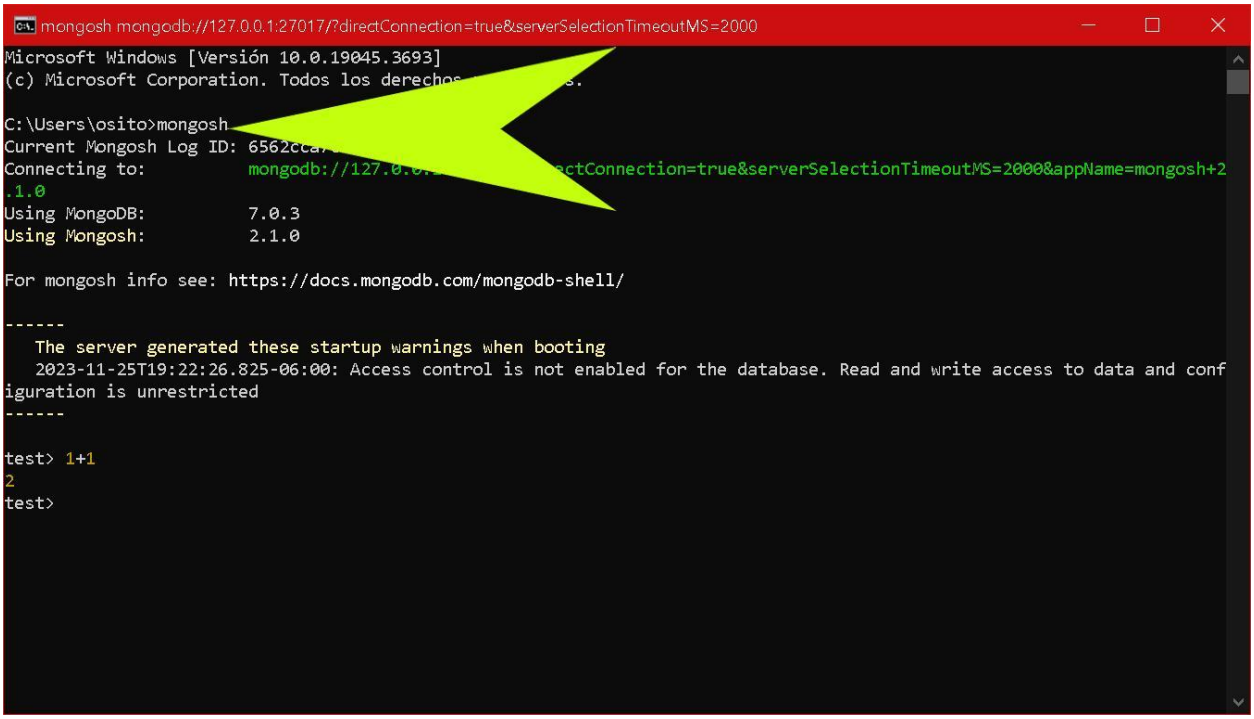
Step 5: Install MongoDB Shell (mongosh)

Important

MongoDB Server and MongoDB Shell are two different things.

- mongod = Runs the MongoDB database server.
- mongosh = Allows you to communicate with the database using commands.
- MongoDB Compass = Graphical interface for managing databases.

The MongoDB MSI installer does not automatically install mongosh, so install it separately.



1. Visit the official download page:

[Download MongoDB Shell]

2. Select Windows. 3. Select the MSI installer. 4. Download and run the installer. 5. Follow the installation instructions.

After installation, open a NEW Command Prompt.

Type:

bash

```
mongosh --version
```

If you see a version number, installation was successful.

Step 6: Check Whether MongoDB Is Running

Now let's verify that MongoDB is running.

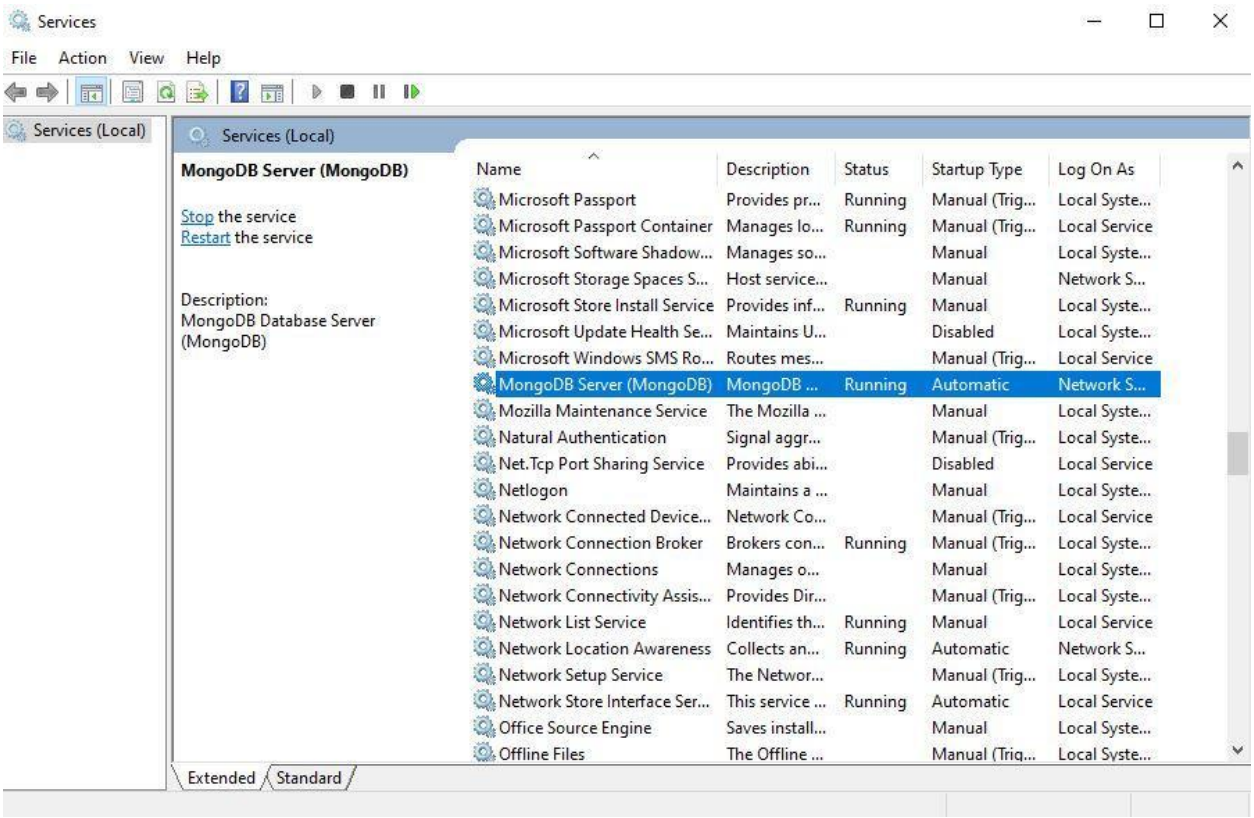
Method 1: Using Windows Services

1. Press Win + R.
2. Type:

text

```
services.msc
```

3. Press Enter.
4. Find MongoDB Server or MongoDB in the services list.
5. Check its status.



Expected result Running

Method 2: Using Command Prompt

Open Command Prompt as Administrator and type:

cmd

net start MongoDB

If MongoDB is already running, Windows may tell you that the service has already been started.

To stop MongoDB:

cmd

net stop MongoDB

To restart it:

cmd

net stop MongoDB

net start MongoDB

Step 7: Connect to MongoDB

Open Command Prompt or PowerShell.

Type:

bash

mongosh

MongoDB Shell should connect to your local MongoDB server.

You should see output similar to:

text

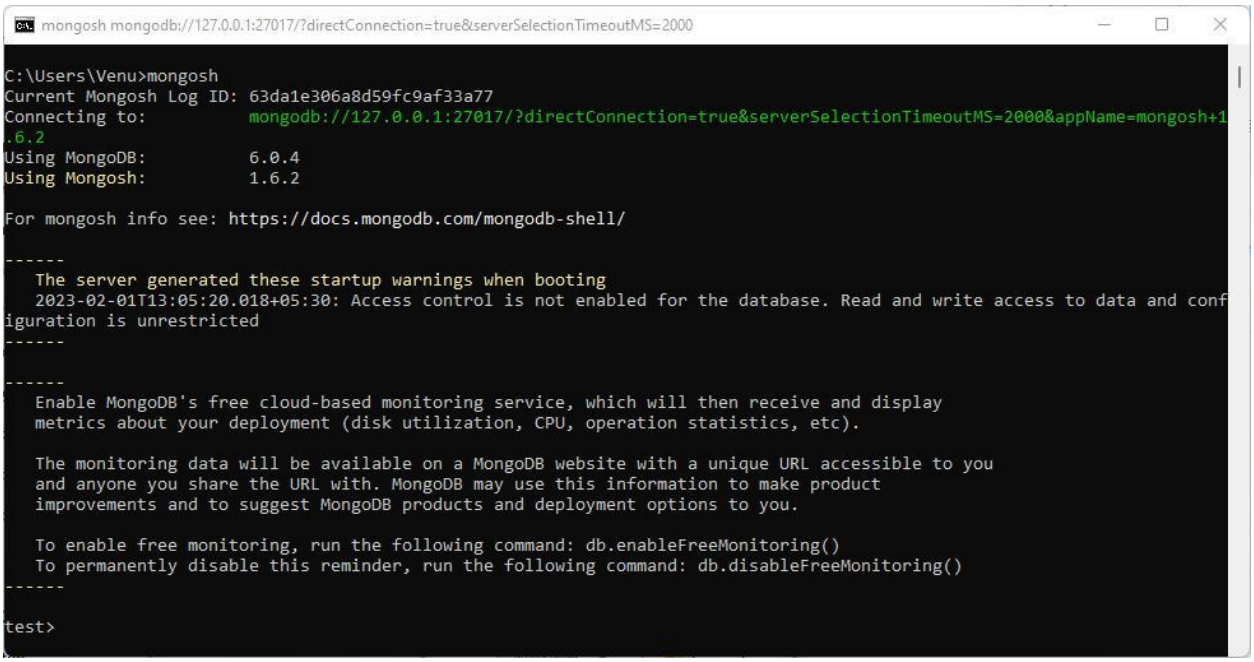
Current Mongosh Log ID: ...

Connecting to: mongodb://127.0.0.1:27017/

Using MongoDB: ...

Using Mongosh: ...

test>



Congratulations! Your MongoDB installation is working.

The default local connection address is:

text

mongodb://localhost:27017

Port 27017 is MongoDB's default port.

Step 8: Create Your First Database

Now let's practice some MongoDB commands.

Inside mongosh, type the following commands.

1. Show all databases

javascript

show dbs

2. Create or switch to a database

javascript

use school

This selects a database called school. MongoDB creates it physically when you first store data in it.

3. Check the current database

javascript

db

Expected output:

text

school

4. Create a collection

javascript

db.createCollection("students")

Expected output:

javascript

{ ok: 1 }

5. Insert your first document

javascript

```
db.students.insertOne({
   name: "Ali",
   age: 22,
   course: "Computer Science"
})
```

6. Display your data

javascript

db.students.find()

Expected result:

javascript

```
[
  {
```

```
  _id: ObjectId("..."),
  name: 'Ali',
  age: 22,
  course: 'Computer Science'
 }
]
```

7. Insert multiple students

```
javascript
db.students.insertMany([
  {
    name: "Ahmed",
    age: 23,
    course: "IT"
  },
  {
    name: "Sara",
    age: 21,
    course: "Software Engineering"
  }
])
```

8. Find a specific student

```
javascript
db.students.findOne({
  name: "Ali"
})
```

9. Update a student

javascript

```
db.students.updateOne(
   { name: "Ali" },
   { $set: { age: 23 } }
)
```

10. Delete a student

javascript

```
db.students.deleteOne({
   name: "Ahmed"
})
```

11. Show collections

javascript

```
show collections
```

12. Show databases

javascript

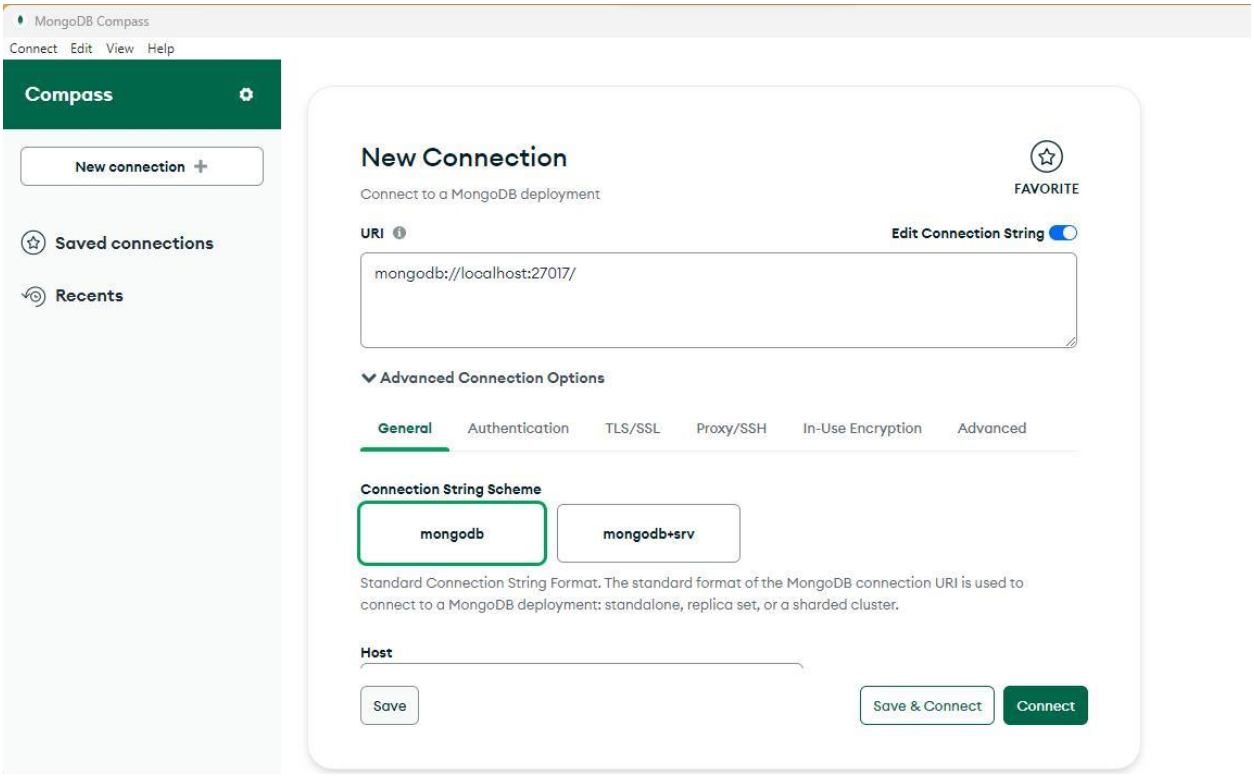
```
show dbs
```

You have now practiced the basic MongoDB CRUD operations:

Operation	MongoDB command
Create	insertOne()
Read	find()
Update	updateOne()
Delete	deleteOne()

Step 9: Connect Using MongoDB Compass

Now let's use MongoDB Compass instead of the command line.

1. 

2. Open MongoDB Compass.
3. Find the connection field.
4. Enter this connection string:

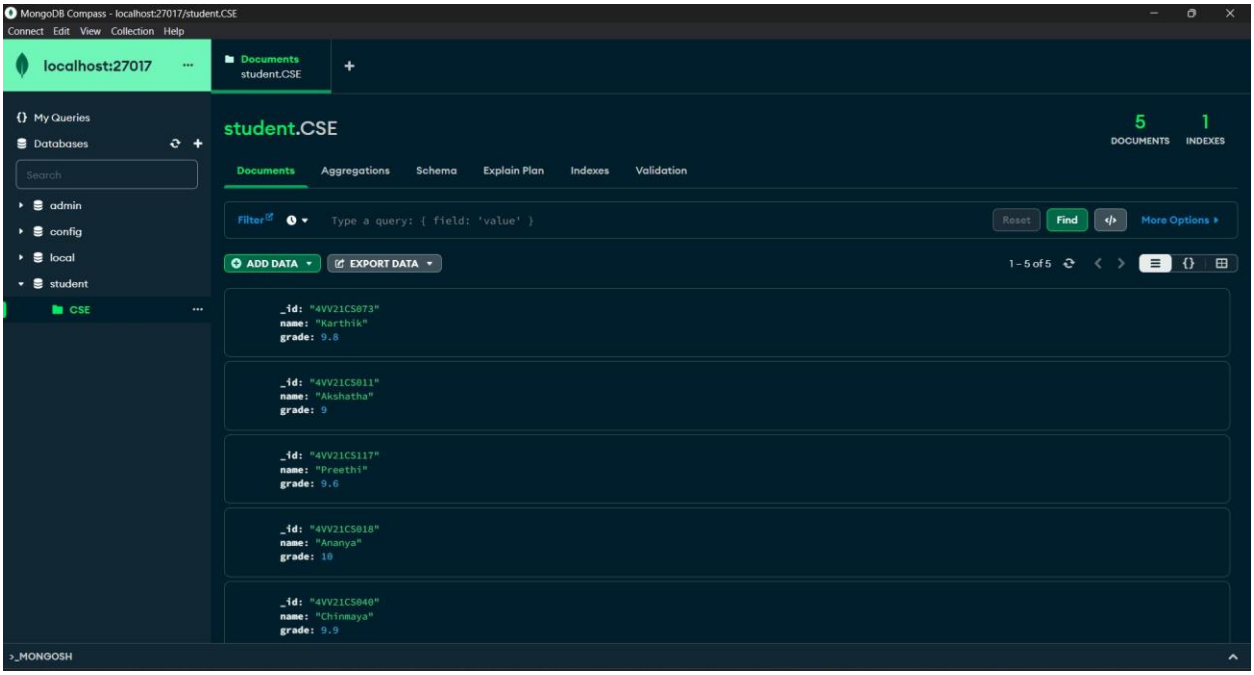
text

mongodb://localhost:27017

4. Click Connect.

After connecting:

- Click Databases.
- Find your school database.
- Open the students collection.
- View the documents you inserted.



You can now create, view, edit, and delete documents using a graphical interface.

Step 10: Add MongoDB to Windows PATH (Optional)

If you type mongod or mongosh and Windows says the command is not recognized, you may need to configure PATH.

For MongoDB Server:

The default installation directory looks like this:

text

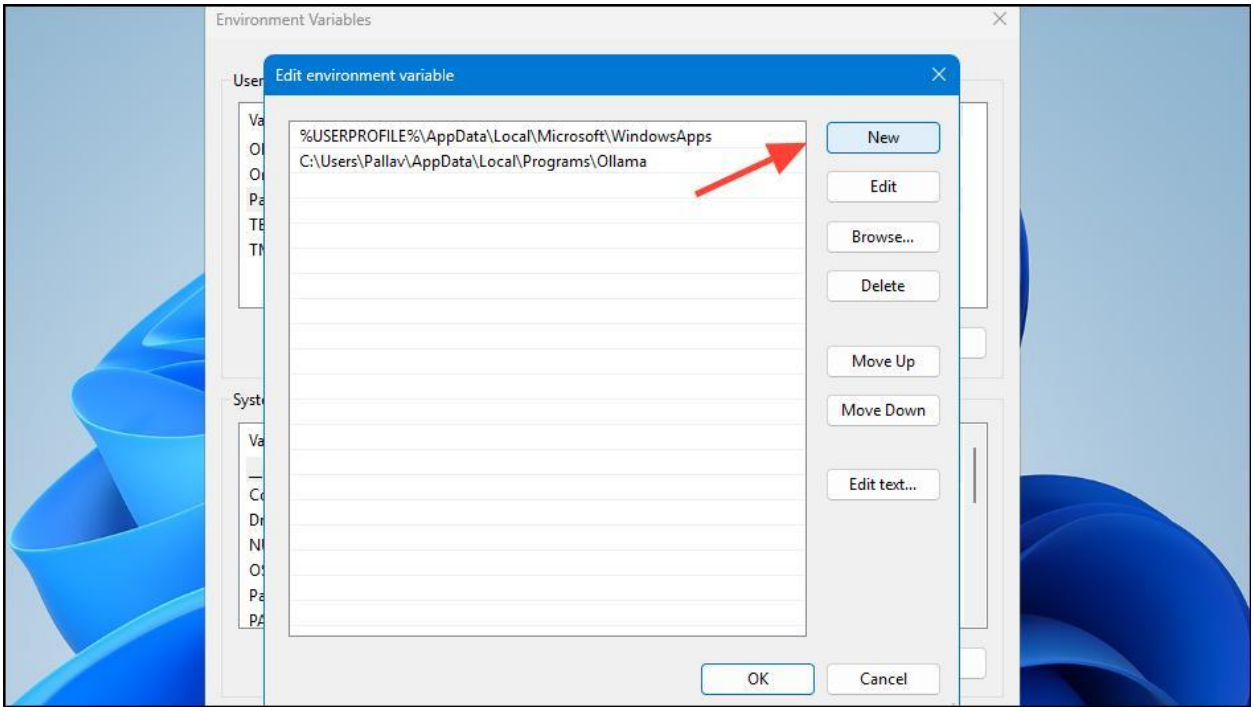
C:\Program Files\MongoDB\Server\8.0\bin

Replace 8.0 with your installed version.

How to add it:

1. Open Windows Search.
2. Search for Environment Variables.
3. Click Edit the system environment variables.

4. Click Environment Variables.
5. Under System variables, select Path.
6. Click Edit.
7. Click New.
8. Paste your MongoDB Server bin directory.
9. Click OK on all windows.



Repeat for the MongoDB Shell installation directory if it is not already in PATH.

Close and reopen Command Prompt.

Test:

cmd

mongod --version

mongosh --version

Both commands should display version information.

Step 11: Common MongoDB Errors and Solutions

Here are some common problems beginners encounter.

Error 1: mongosh is not recognized

Command not recognized

text

'mongosh' is not recognized as an internal

or external command

Solution:

- Make sure MongoDB Shell is installed.
- Add its installation directory to PATH.
- Open a new Command Prompt.

Error 2: MongoDB connection refused

text

MongoNetworkError: connect ECONNREFUSED

127.0.0.1:27017

Solution:

Your MongoDB server may not be running.

Open Command Prompt as Administrator:

cmd

net start MongoDB

Then reconnect:

bash

mongosh

Error 3: MongoDB service does not exist

text

The service name is invalid.

Solution:

- Press Win + R.
- Enter services.msc.
- Check whether MongoDB is installed as a service.
- If it is missing, rerun the MongoDB installer and enable service installation.

Error 4: Port 27017 is already in use

Another process may already be using MongoDB's default port.

Check using:

cmd

netstat -ano | findstr :27017

Check which process owns the port before stopping or changing anything.

Error 5: Database does not appear

If you run:

javascript

use school

and then:

javascript

show dbs

You might not see school.

This is normal. MongoDB does not persist an empty database. Insert a document first:

javascript

```
db.students.insertOne({
   name: "Ali"
})
```

Then:

javascript

show dbs

Step 12: Important MongoDB Commands Cheat Sheet

Keep these commands for future use.

Command	Purpose
mongosh	Connect to MongoDB
show dbs	Show databases
use school	Switch database
db	Current database
show collections	Show collections
db.createCollection("users")	Create collection
db.users.insertOne({name:"Ali"})	Insert document
db.users.find()	Retrieve documents
db.users.findOne()	Retrieve one document
db.users.updateOne()	Update document
db.users.deleteOne()	Delete document
db.users.drop()	Delete collection
db.dropDatabase()	Delete current database

Warning: db.dropDatabase() permanently deletes the current database's data. Use it carefully.

Final Installation Checklist

Your installation progress

0/10 completed

Downloaded MongoDB Community Server MSI

Installed MongoDB using Complete setup

Enabled MongoDB Windows Service

Installed MongoDB Compass

Installed MongoDB Shell (mongosh)

Verified mongosh --version

Successfully connected using mongosh

Connected MongoDB Compass to localhost:27017

Created school database

Inserted and retrieved student records

Official MongoDB Resources

- MongoDB Official Documentation
- MongoDB Windows Installation Guide
- MongoDB Shell Documentation
- MongoDB Compass Documentation
- MongoDB University – Free Learning

These installation steps follow MongoDB's official Windows installation documentation.

MongoDB Docs

+2

What should you learn next?

I recommend learning MongoDB in this order:

1. MongoDB Database, Collections, and Documents.
2. CRUD Operations.
3. MongoDB Query Operators.
4. Sorting, Filtering, and Pagination.

5. Aggregation Pipeline.
6. MongoDB with Node.js.
7. MongoDB with Express.js.

My recommendation: Once MongoDB is installed, learn CRUD operations and then connect MongoDB with Node.js. This will help you build real-world backend projects.